RICE UNIVERSITY

Distributed Matrix Multiplication in 2D

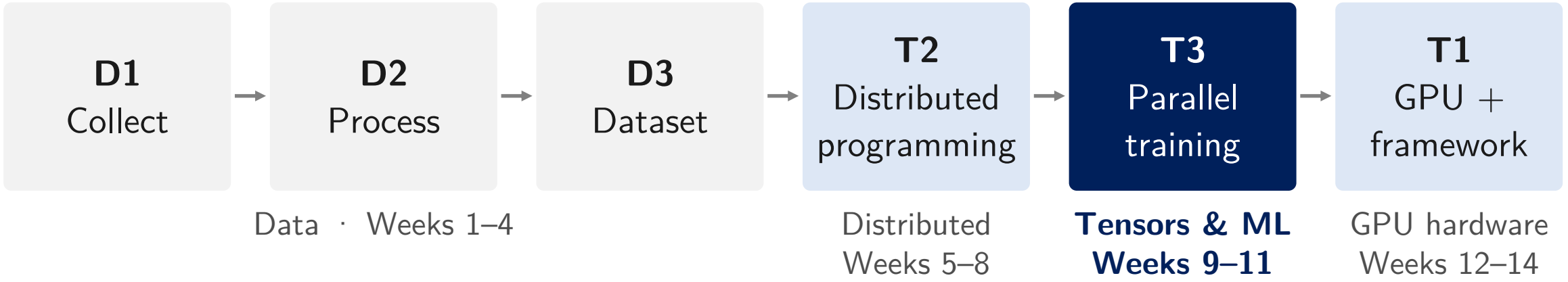
Blocks on a processor grid: Cannon's algorithm, SUMMA, and what they send

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 11, Lecture 1

Where we are in the pipeline



In our pipeline

Today: multiply matrices that live on **many processors**, and count what they send. **T3 checkpoint** this week; Lab 5 due after Lecture 2.

Week 10: split a layer by columns or rows → today: split a matmul on a 2D grid → Lecture 2: 3D

Why do we need distributed matrix multiply?

The job: $C = A \cdot B$, two $n \times n$ matrices, $n = 65{,}536$ (BF16)
on **64 GPUs**; each matrix ≈ 8.6 GB

Each GPU computes 1/64 of C. Which takes longer: the arithmetic, or moving the data?



Answer

Moving data takes $\approx$ **19×** the arithmetic. The **layout** of the matrices sets the speed.

Source: NVIDIA DGX H100 User Guide (400 Gb/s per GPU); H100 datasheet (SXM5), BF16 peak. 50 GB/s per GPU over the network.

Distributed matrix multiply, in plain words

a 3×3 processor grid

P(0,0)	P(0,1)	P(0,2)
P(1,0)	P(1,1)	P(1,2)
P(2,0)	P(2,1)	P(2,2)

- **Grid**: p processors in $\sqrt{p}$ rows $\times$ $\sqrt{p}$ columns.
- **Block**: each matrix cut into $\sqrt{p} \times \sqrt{p}$ tiles; P(i, j) stores tile (i, j) of A, B and C.

P(1, 2) computes C_{12} — but it needs tiles of A and B that **others** own.

Key idea

An algorithm decides **who sends which tile to whom, and when**. Today: Cannon and SUMMA.

Where it shows up

- **1969, Cannon**: a grid of processors that talk only to their **neighbours**.
- **1997, SUMMA**: **broadcasts** along rows and columns; any grid, any shape.
- **Deep learning**: Week 10's tensor parallelism is a 1D layout; 2D versions use SUMMA.
- **In this course**: T3 and Lab 5 use 1D layouts; Lecture 2 places them.

In our pipeline

Why distributed matmul? The matrices of large models live on **many GPUs**, and the communication pattern sets the speed.

Source: Cannon, PhD thesis, 1969; van de Geijn & Watts, Concurrency: Practice and Experience, 1997.

Learning goals for today

- **Explain** why matrix multiply is the unit of ML work, and write it with **blocks**.
- **Compare** 1D and 2D layouts: what each processor must receive.
- **Run** Cannon's algorithm (skew, then multiply-and-shift) and SUMMA (broadcast panels).
- **Count** messages and words per processor and cost them with $\alpha + \mathbf{m}/\beta$.

A puzzle to start

Layout 1 — strips
64 processors,
each holds n/64 rows

Layout 2 — a grid
8×8 processors,
each holds one $(n/8) \times (n/8)$ block

Same work per processor. How much data must each one receive in each layout?

Answer

Strips: $\approx \mathbf{n^2}$ numbers (all of B). Grid: $\mathbf{n^2/4}$ — $4\times$ less, and the gap grows with p.

Recap: messages and their cost

$$\text{one message of } m \text{ bytes:} \quad T = \alpha + m / \beta$$

$$A\,(M \times K) \cdot B\,(K \times N)\text{:} \quad 2 \cdot M \cdot N \cdot K \text{ FLOPs}$$

Weeks 6–7

α: start-up cost per **message**. β: bandwidth. Today we count **messages** and **words** per processor.

Part 1

Matrix multiply, in blocks

What must each processor receive?

Matrix multiply is the unit of ML work

Hidden layer
$(64 \times 128) \cdot (128 \times 64)$

Logits
$(64 \times 64) \cdot (64 \times 322)$

Matrix products: $\approx$ **97%** of the forward arithmetic of our MLP.

- Backward: **two more** matrix products per layer.
- Large models: the same pattern, much wider matrices.

Key idea

Fast training on many GPUs $\approx$ a fast **distributed matrix multiply**.

Work grows faster than data

$$\text{work: } 2n^3 \text{ FLOPs}$$

$$\text{data: } 3n^2 \text{ numbers}$$

$$\text{work per number: } \approx n$$

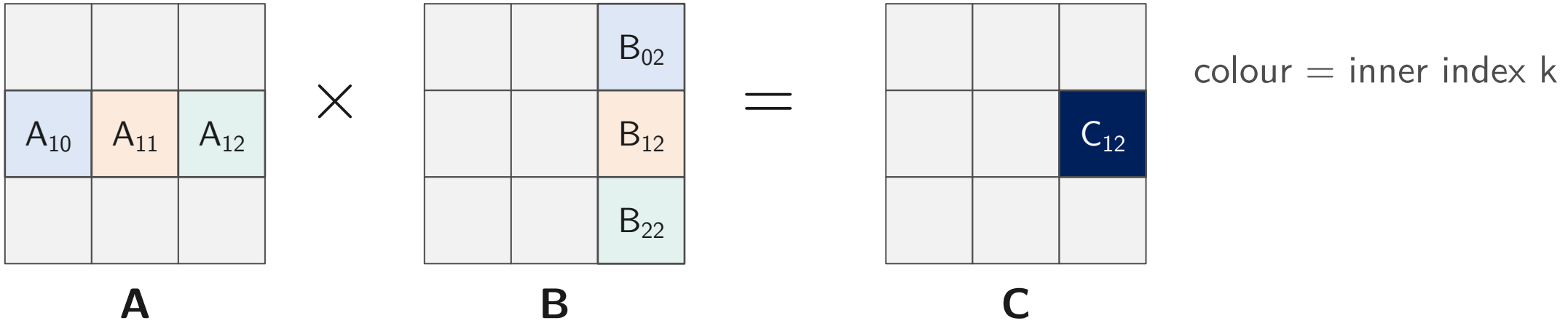
Every number of A is used **n times**:

once for every column of B.

Key idea

Reuse is what we exploit: keep big blocks **local**, send as few numbers as possible.

Block matrices: one formula



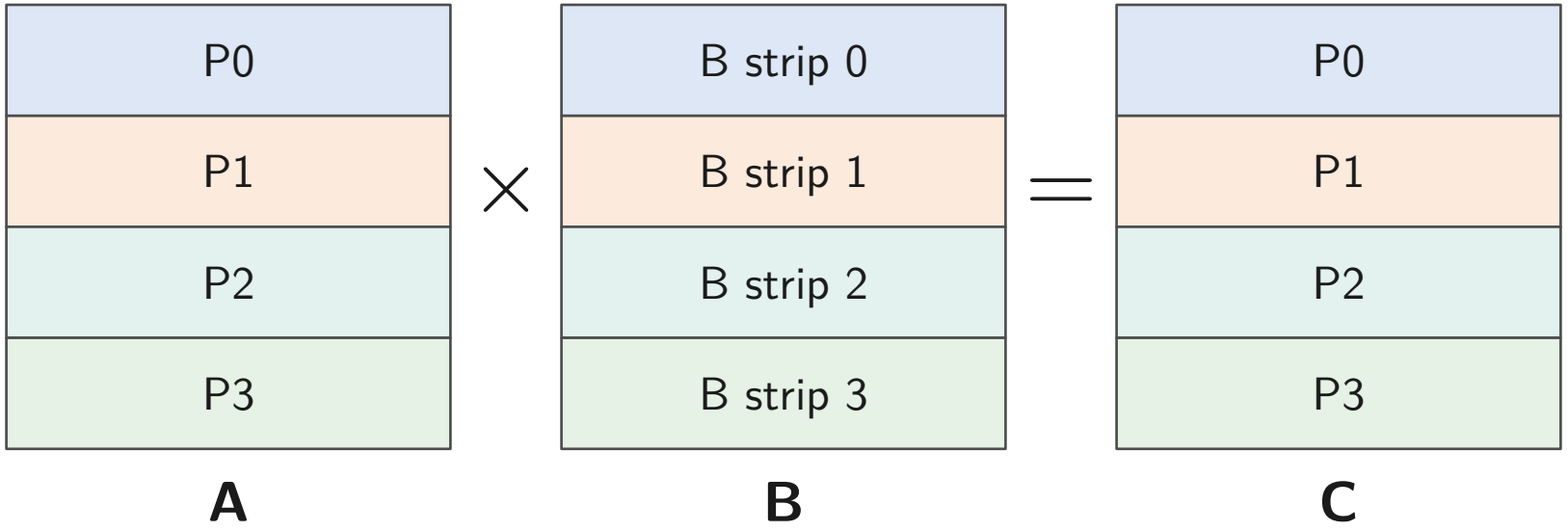
colour = inner index k

$$C_{12} = A_{10}B_{02} + A_{11}B_{12} + A_{12}B_{22}$$

Key idea

A block of A meets a block of B only when their **inner index k** matches.

1D layout: strips of rows



Processor r owns **row strip r** of A, B and C.

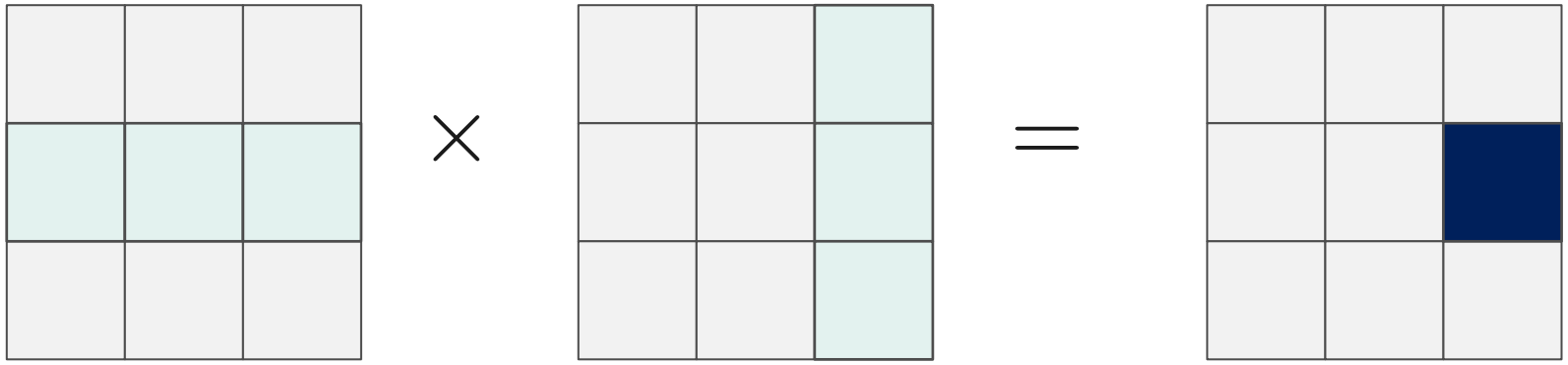
Its rows of A meet **all of B**.

The B strips travel around a **ring**: $p - 1$ steps.

Watch out

1D: each processor receives $\approx \mathbf{n^2}$ numbers — all of B, **whatever p is**.

2D layout: a $\sqrt{p} \times \sqrt{p}$ grid



P(1, 2) needs **block row 1** of A

and **block column 2** of B:

$2\sqrt{p}$ blocks of n^2/p numbers.

$$\text{received} \approx 2\sqrt{p} \cdot n^2/p = 2n^2 / \sqrt{p}$$

Key idea

2D: words per processor **fall like $1/\sqrt{p}$**; 1D stays at $\approx n^2$. Ratio $\approx \sqrt{p} / 2$.

Check yourself

Two minutes with a neighbour.

- How many FLOPs does an n $\times$ n matmul take, and how often is each number of A used?
- p = 16: how many numbers must each processor receive in 1D? In a 4 $\times$ 4 grid?
- On a 3 $\times$ 3 grid, write C_{20} as a sum of block products. Which blocks does P(2, 0) need?

Part 2

Cannon's algorithm

Rotate the blocks so that every processor always has a matching pair

Cannon's idea: shift, don't copy

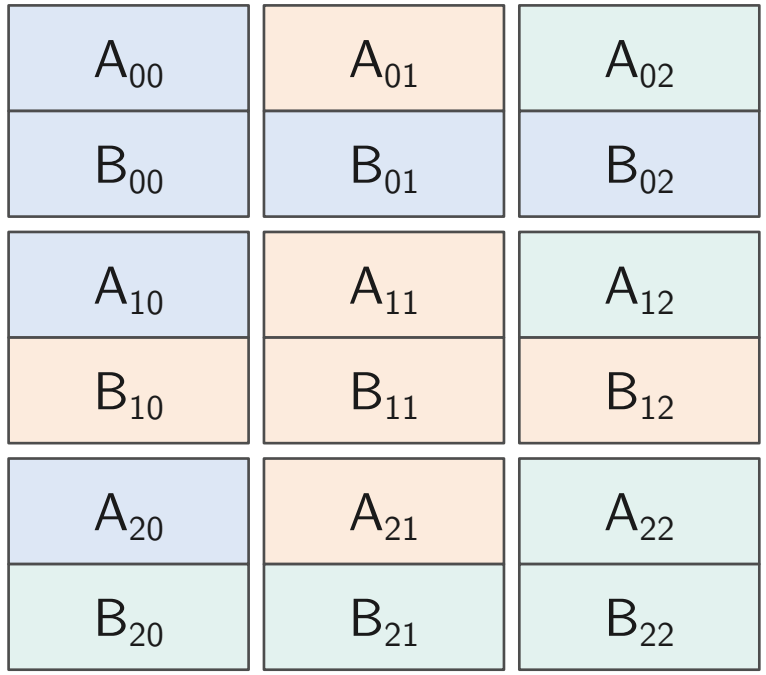
- P(i, j) needs A(i,k) B(k,j) for **every k**, but only **one pair at a time**.
- Keep **one** A block and **one** B block per processor; **move** them each step.
- A blocks move **left** along grid rows; B blocks move **up** along grid columns (with wraparound).

Key idea

Memory stays at **three blocks** per processor, and every message goes to a **neighbour**.

Source: L. E. Cannon, PhD thesis, Montana State University, 1969.

The starting layout



P(i, j) holds A(i,j) (top) and B(i,j) (bottom)

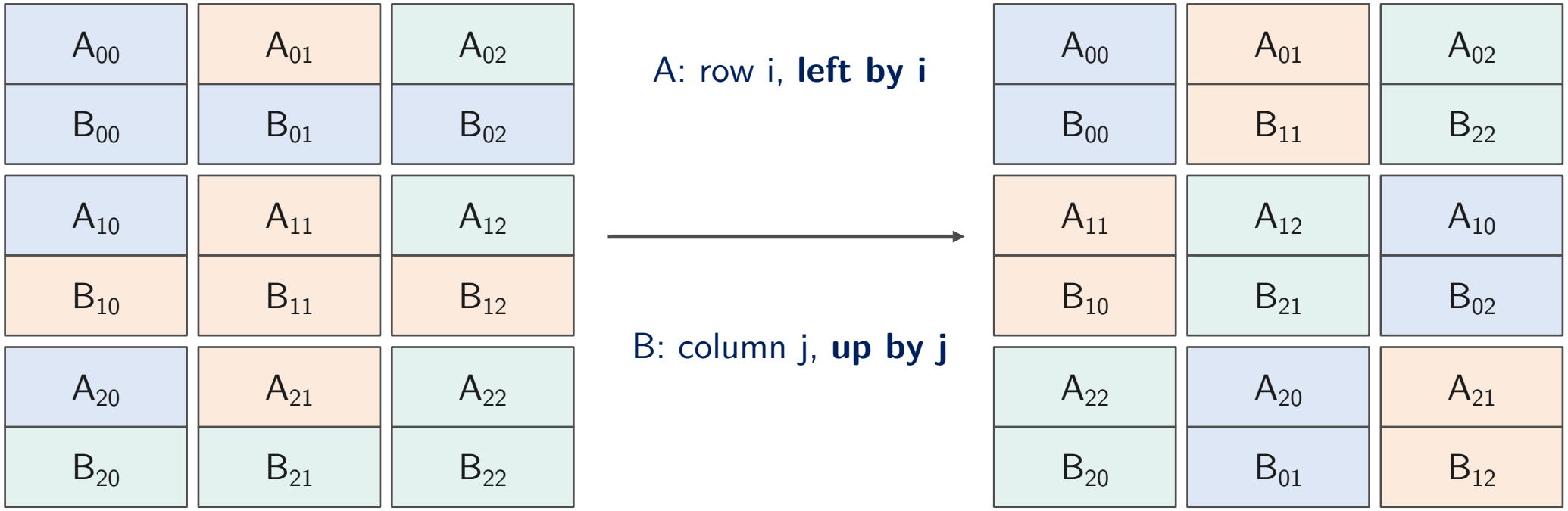
colour = inner index k: blue 0 · orange 1 · green 2

Which processors hold a matching pair (same colour)?

Key idea

Fix it once, before the loop: **skew** the blocks.

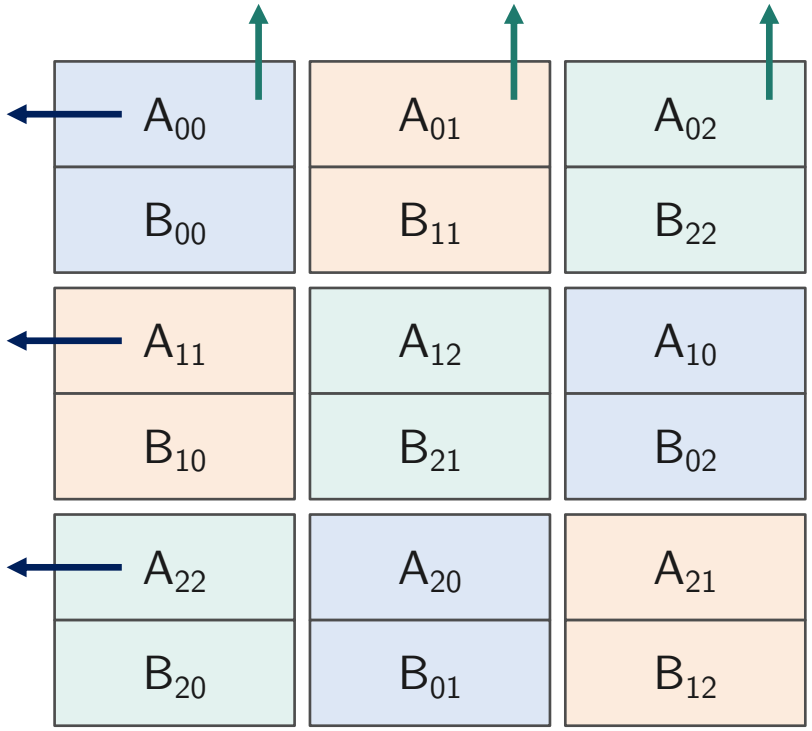
The skew



Check

After the skew P(i, j) holds A(i,k) and B(k,j) with $\mathbf{k = (i + j) \bmod \sqrt{p}}$: every pair matches.

One step: multiply, then shift



1. Multiply

C(i,j) += A(i,k) B(k,j)

on every processor, no messages

2. Shift

every A block one place **left**,

every B block one place **up**

Key idea

Both k's go up by one, so the pairs **still match**. Per step: 2 messages per processor.

$\sqrt{p}$ steps cover every k

step t = 0

0	1	2
1	2	**0**
2	0	1

step t = 1

1	2	0
2	0	**1**
0	1	2

step t = 2

2	0	1
0	1	**2**
1	2	0

each cell: the k that P(i, j) multiplies; bold = P(1, 2)

Check

After $\sqrt{p}$ steps every P(i, j) has used **every k exactly once**: C(i,j) is complete.

Cannon in mpi4py

```
cart = comm.Create_cart([q, q], [True, True])
R, L = cart.Shift(1, -1)   # right, left
D, U = cart.Shift(0, -1)   # down, up
for t in range(q):
    C += A @ B             # local product
    if t < q - 1:          # shift for next step
        cart.Sendrecv_replace(A, L, source=R)
        cart.Sendrecv_replace(B, U, source=D)
```

- A grid **with wraparound**.
- **Shift**: who is left, right, up, down.
- **Sendrecv_replace**: send and receive in one call — no deadlock.

Key idea

The skew uses the same calls, shifting A by i and B by j places.

Source: mpi4py documentation: Cartcomm.Shift, Comm.Sendrecv_replace.

Counting Cannon's communication

$$\text{messages:}\quad 2 + 2(\sqrt{p} - 1) = 2\sqrt{p}$$

$$\text{words:}\quad 2\sqrt{p} \cdot n^2/p = 2n^2 \,/\, \sqrt{p}$$

$$\text{memory:}\quad \approx 3n^2/p$$

Check

Cannon sends **exactly** what Part 1 said a processor must receive — and stores only three blocks.

Check yourself

Two minutes with a neighbour.

- Why can P(i, j) not start multiplying before the skew?
- 3×3 grid, step $t = 1$: which A and B blocks does P(2, 1) hold?
- $p = 16$: how many messages and words does one processor send in Cannon?

Part 3

SUMMA and the cost of 2D

Broadcast panels instead of shifting — and what does 2D save?

What Cannon needs

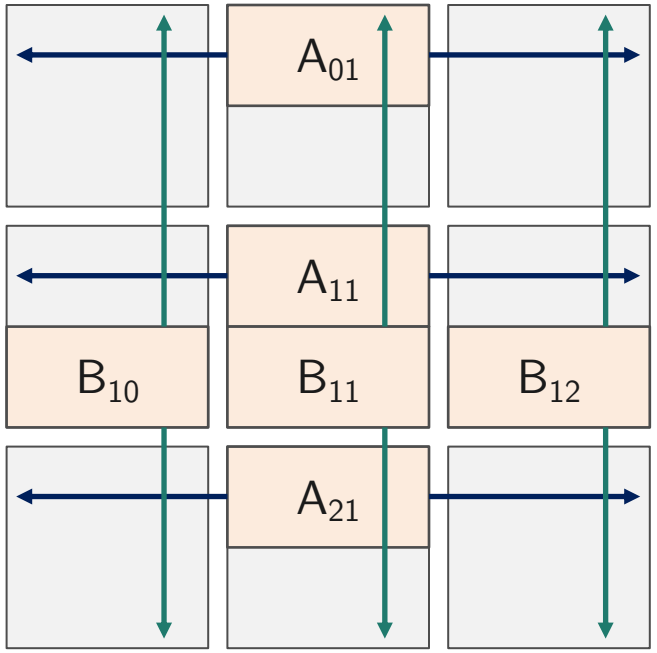
- a **square** grid: p must be a perfect square;
- square, equal **blocks**, and a **skew** before it starts;
- but ML matrices are **rectangular** (weights h $\times$ 4h, activations b $\times$ h).

Key idea

SUMMA keeps the 2D grid, replaces shifts by **broadcasts**, and works for **any grid and shape**.

Source: van de Geijn & Watts, "SUMMA: Scalable Universal Matrix Multiplication Algorithm," Concurrency: Practice and Experience 9(4), 1997.

SUMMA: one step k



Broadcast A(i,k) along each grid row

Broadcast B(k,j) along each grid column

Multiply locally:
C(i,j) += A(i,k) B(k,j)

then k = k + 1, $\sqrt{p}$ steps

Key idea

SUMMA = for each k: **broadcast** A(i,k) along rows, B(k,j) along columns, **multiply**.

Source: van de Geijn & Watts, "SUMMA: Scalable Universal Matrix Multiplication Algorithm," Concurrency: Practice and Experience 9(4), 1997.

SUMMA in mpi4py

```
row = cart.Sub([False, True])   # my grid row
col = cart.Sub([True, False])   # my grid column
for k in range(q):
    a = A.copy() if j == k else np.empty_like(A)
    row.Bcast(a, root=k)
    b = B.copy() if i == k else np.empty_like(B)
    col.Bcast(b, root=k)
    C += a @ b
```

- **Sub**: one communicator per grid row, one per column.
- **Bcast**: the owner sends, the rest receive.

Key idea

No skew and no wraparound: just **two broadcasts and one multiply** per step.

Source: mpi4py documentation: Cartcomm.Sub, Comm.Bcast.

SUMMA versus Cannon

Per processor	Cannon	SUMMA
words received	$2n^2/\sqrt{p}$	$\approx 2n^2/\sqrt{p}$
communication	shifts to neighbours	broadcasts along rows/columns
grid and shapes	square only	any

Key idea

Same words; SUMMA trades a little latency (tree broadcasts) for **flexibility**.

Source: van de Geijn & Watts, "SUMMA," 1997. Tree broadcast: Week 7, Lecture 2.

The cost in the α–β model

$$1D: \quad T \approx (p-1)\,\alpha \;+\; n^2 \cdot w \,/\, \beta$$

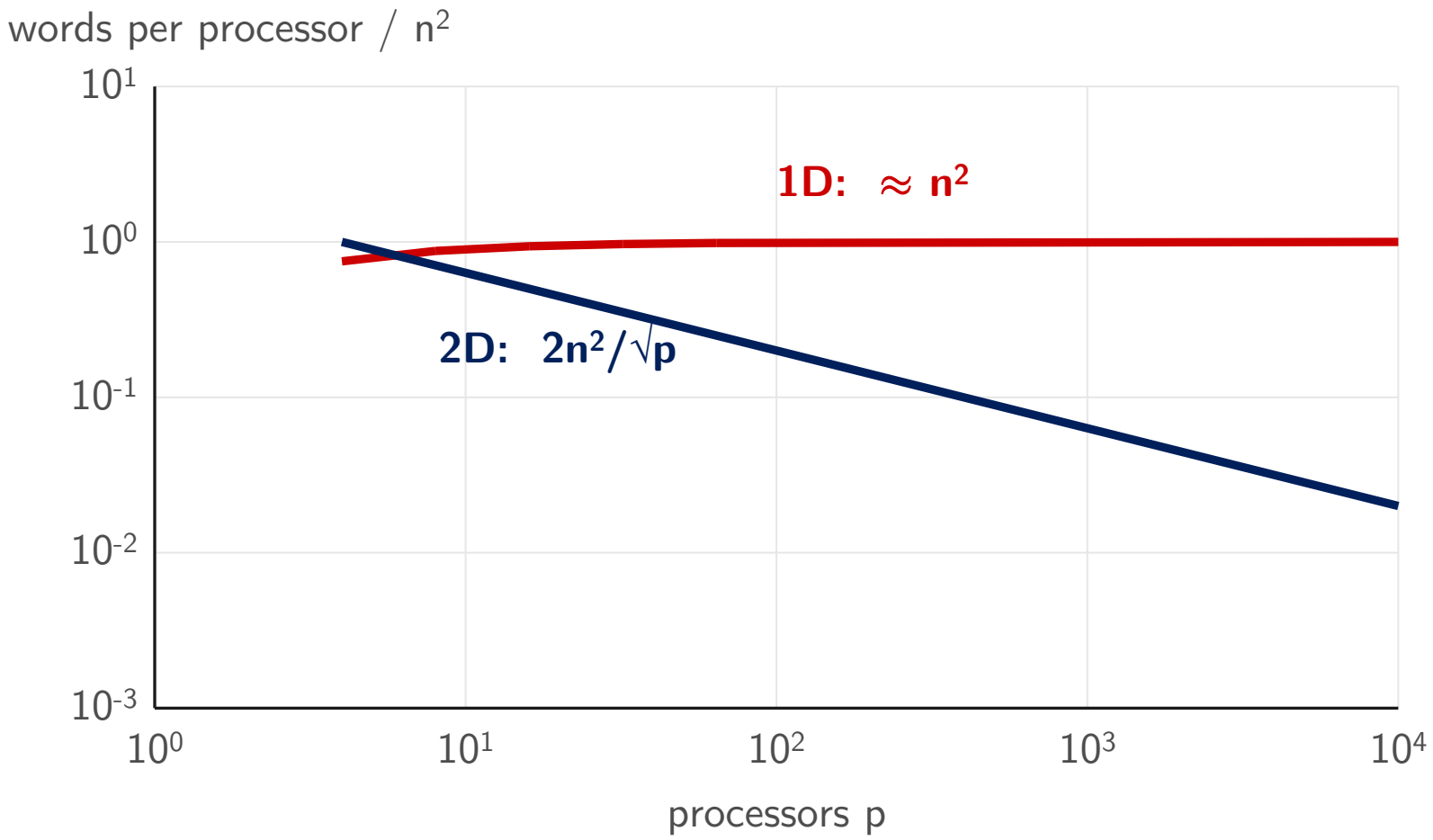
$$2D: \quad T \approx 2\sqrt{p}\,\alpha \;+\; (2n^2 \,/\, \sqrt{p}) \cdot w \,/\, \beta$$

- **Latency** term: $p - 1$ messages $\to 2\sqrt{p}$.
- **Bandwidth** term: $n^2 \to 2n^2/\sqrt{p}$ words, $\sqrt{p}/2$ times fewer.

Memory

w: bytes per number. Large matrices: the **bandwidth** term dominates, and 2D cuts it by $\sqrt{p}/2$.

Words per processor as p grows

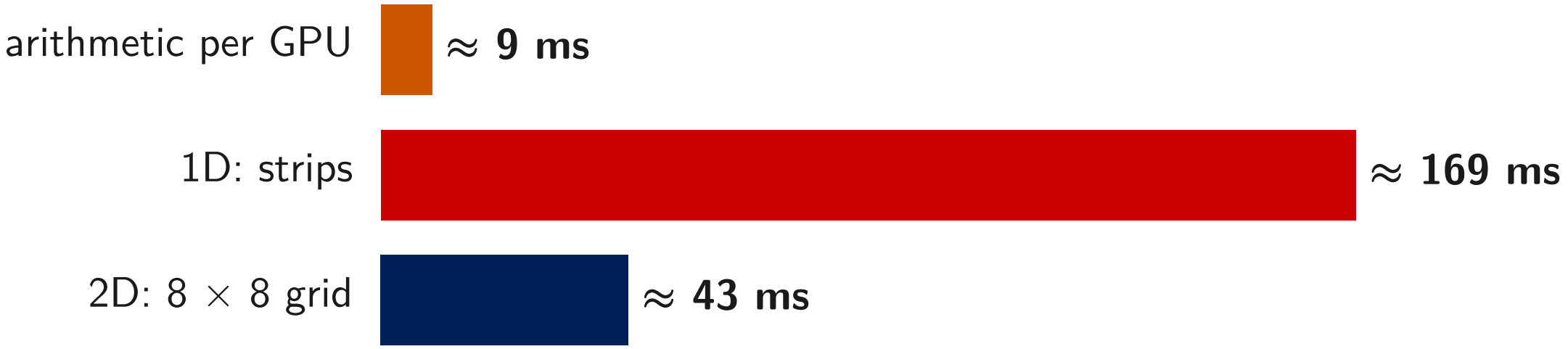


p = 64: **4×** fewer

p = 4,096: **32×** fewer

Lecture 2: a third curve.

Back to the opening example



Key idea

2D cuts the transfer **4×** — still above the arithmetic. Lecture 2: spend **memory** to send even less.

Source: NVIDIA DGX H100 User Guide (400 Gb/s per GPU); H100 datasheet (SXM5), BF16 peak. Bandwidth terms only.

Check yourself

Two minutes with a neighbour.

- Cannon and SUMMA move the same words. Why do libraries prefer SUMMA?
- Going from 64 to 1,024 processors: how do words per processor change in 1D? In 2D?
- How many times fewer words does 2D send than 1D at $p = 16$? At $p = 4{,}096$?

IN-CLASS BUILD

Run Cannon by hand

Work in pairs: a 3×3 grid, one number per block. Track the blocks, then count.

Exercise: Cannon on a 3 × 3 grid

1	0	2
3	1	0
0	2	1

A

2	1	0
0	1	3
1	0	2

B

- (1) After the skew, which a and b does **P(1, 2)** hold at t = 0, 1, 2?
- (2) Compute $\mathbf{c_{12}}$ step by step; check it against A · B.
- (3) If each block were b × b: messages and words **P(1, 2)** sends; messages in total.
- (4) The same counts for a **1D ring** on 9 processors.

USE

k = (i + j + t) mod 3. Skew: A row i left by i, B column j up by j.

STRETCH

Which processors send no skew message at all?

Solutions: tracking the blocks

step t	k	P(1, 2) holds	c_{12} so far
0	0	$a_{10} = 3,\ \ b_{02} = 0$	0
1	1	$a_{11} = 1,\ \ b_{12} = 3$	3
2	2	$a_{12} = 0,\ \ b_{22} = 2$	**3**

Check

$c_{12} = 3$, the same as row 1 of A times column 2 of B: every k used **once**.

Solutions: counting the messages

9 processors	P(1, 2) messages	P(1, 2) words	all messages
Cannon, 3×3	6	$6b^2$	48
1D ring	8	$8b^2$	72

$$\text{Cannon: } 2\sqrt{p} \text{ messages}, \quad 2n^2/\sqrt{p} = 2 \cdot 9b^2 \,/\, 3 = 6b^2 \text{ words}$$

Check

At p $= 9$ the saving is small ($6b^2$ vs $8b^2$); it grows like $\sqrt{\mathbf{p}}$ **/ 2**. Stretch: P(0, 0) skips the skew.

Key takeaways

- Matrix multiply is **most of the arithmetic** of training; each number is reused n times.
- Blocks: **C(i,j) = Σ over k of A(i,k) B(k,j)**; a pair of blocks multiplies only when k matches.
- 1D strips: every processor receives ≈ $\mathbf{n^2}$ numbers, whatever p is.
- Cannon: **skew**, then $\sqrt{p}$ × (multiply, shift); $\mathbf{2\sqrt{p}}$ messages, $\mathbf{2n^2/\sqrt{p}}$ words, 3 blocks of memory.
- SUMMA: **broadcast** panels along rows and columns; same words, any grid and shape.

Next time, and readings

When	What
Lecture 2 (this week)	3D matrix multiplication and tensor parallelism
This week	T3 checkpoint; Lab 5 due after Lecture 2
Week 12	GPU architecture; T3 due Friday

READ BEFORE LECTURE 2

van de Geijn & Watts, "SUMMA: Scalable Universal Matrix Multiplication Algorithm," 1997 (§1–3).

Cannon, 1969 — or any textbook description of Cannon's algorithm.